

FaceFusion

Industry-leading face manipulation platform — a source-level architecture reference

v3.6.1

VERSION

ONNX

RUNTIME 1.24

11

PROCESSORS

4

UI LAYOUTS

9

EXEC PROVIDERS

Subject: FaceFusion codebase — end-to-end architecture, subsystems & data flow

Basis: Direct source analysis of the repository (CLI, workflows, processors, inference, jobs, UI)

License / Author: OpenRAIL-AS · Henry Ruhs · facefusion.io

Runtime: Python 3.10+ · ONNX Runtime · OpenCV · FFmpeg · Gradio 5.44

Table of Contents

1. System Overview & Technology Stack

What FaceFusion is, the runtime stack, and the layered architecture at a glance

2. Entry Point & CLI Dispatch

facefusion.py → core.py, command routing, pre-checks, headless/batch flows

3. Program, Arguments & State

program.py composition, args bridging, dual CLI/UI state, config & choices

4. The Face Analysis Pipeline

Detection → landmarking → recognition → classification → selection → masking

5. The Processor Plugin System

Dynamic loading, the 8-method contract, process_frame, and the 11 processors

6. Inference & Execution Layer

ONNX session pooling, execution-provider priority, model download & integrity

7. Processing Workflows & Media I/O

Image & video pipelines, FFmpeg builder, vision, audio, temp lifecycle

8. The Job System

Persisted JSON jobs, CRUD, sequential runner, headless/batch as ephemeral jobs

9. The UI System

Gradio layouts, the render/listen/update component pattern, cross-wiring

10. Cross-Cutting Concerns

Content-safety gate, threading model, caching, memory, appendices

How to read this document

Each section maps to a cluster of modules under **facefusion/**. Inline references use the **file.py:line** convention so you can jump straight to the source. Diagrams show data/control flow; tables enumerate models, options, and interfaces.

SECTION 1

System Overview & Technology Stack

FaceFusion is a face-manipulation platform built around **ONNX Runtime** for portable, hardware-accelerated inference. It performs face swapping, enhancement, editing, age modification, lip-syncing, colorization, upscaling and more, driven either through a command-line interface or a **Gradio** web UI. The same processing core serves both.

1.1 Design principles

- **Plugin-oriented processing.** Every transformation (swap, enhance, edit, ...) is a self-contained *processor* module conforming to a fixed 8-method interface, loaded dynamically by name.
- **Model-agnostic inference.** A central **inference_manager** pools ONNX sessions; execution providers (CUDA, TensorRT, DirectML, ...) are auto-detected and prioritized at runtime.
- **Shared analysers.** Face detection, landmarking, recognition, classification and masking are computed once per frame and reused by all processors through a content-hash cache.
- **Reproducible jobs.** Every run — interactive, headless, or batch — is expressed as a persisted JSON *job* of ordered *steps*, giving identical execution semantics everywhere.
- **Dual state contexts.** CLI and UI keep separate state buckets, auto-selected by call-stack inspection, with explicit sync bridges.

1.2 Technology stack

Concern	Technology	Notes
Language / runtime	Python 3.10+ (CI on 3.12)	Enforced at startup in core.py:95
Inference	ONNX Runtime 1.24.4, ONNX 1.21.0	Session pooling & provider selection
Numerics	NumPy 2.2.1, SciPy 1.17.1	Frame math, STFT/mel for audio
Computer vision	OpenCV 4.13	Warping, masking, video capture
Media I/O	FFmpeg (external), curl (external)	Must be on <code>PATH</code> ; frame extract/merge, audio
Web UI	Gradio 5.44.1 (+ rangeslider 0.0.8)	Layouts & components
Progress / CLI	tqdm 4.67.3	Progress bars for extraction, processing, merge
Model delivery	GitHub Releases & Hugging Face	On-demand download to .assets/models/ with CRC32 verification

1.3 The layered architecture

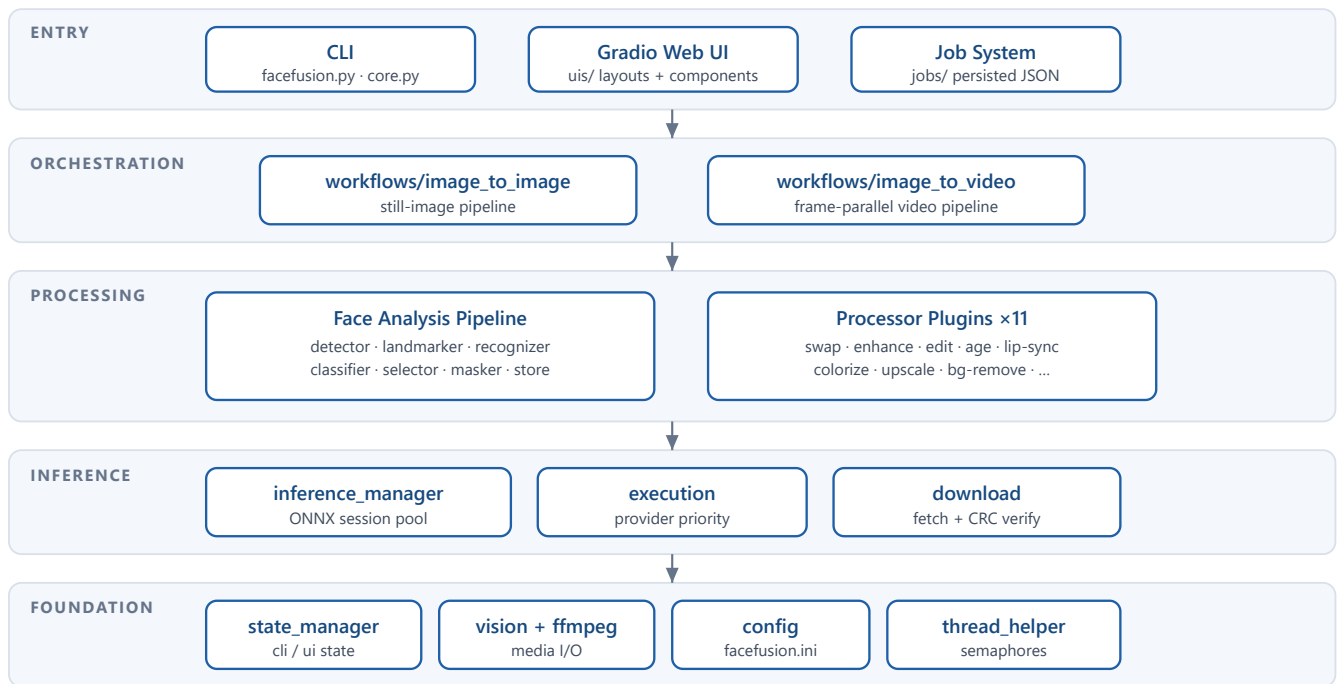


Figure 1 — Five-layer architecture. Both the CLI and the Gradio UI drive the same orchestration → processing → inference stack.

1.4 Repository map

facefusion/core.py	CLI entry, command routing, pre-checks, headless/batch drivers
facefusion/program*.py	ArgumentParser composition & validation
facefusion/args.py	args ↔ state ↔ job/step bridging
facefusion/state_manager.py	dual cli/ui global state container
facefusion/face_*.py	detection, landmarking, recognition, classification, selection, masking, store, analyser
facefusion/processors/	plugin loader + 11 processor modules + pixel_boost / live_portrait
facefusion/inference_manager.py	ONNX session pooling & caching
facefusion/execution.py	execution-provider detection & options
facefusion/workflows/	image_to_image & image_to_video orchestration
facefusion/vision / ffmpeg*	frame & video/media I/O
facefusion/audio / voice_extractor	audio decode, mel features, vocal isolation
facefusion/jobs/	job manager, runner, store, helpers, list
facefusion/uis/	Gradio layouts & components
facefusion/content_analyser.py	NSFW safety gate (hash-pinned)

SECTION 2

Entry Point & CLI Dispatch

The single executable `facefusion.py` defers to `core.py::cli()`, which builds the argument parser, validates it, applies arguments into global state, and routes to a command handler. All heavy commands pass through a two-stage pre-check before any model runs.

2.1 The cli() bootstrap

The bootstrap sequence in `core.py:23–40` is deliberately linear:

```
def cli():
    if pre_check():
        # python ≥ 3.10, curl & ffmpeg on PATH
        signal.signal(SIGINT, signal_exit) # graceful Ctrl-C
        program = create_program() # compose all sub-parsers
        if validate_args(program): # defaults must be valid choices
            args = vars(program.parse_args())
            apply_args(args, state_manager.init_item) # seed BOTH cli+ui buckets
            if state_manager.get_item('command'):
                logger.init(...); route(args)
            else: program.print_help()
        else: hard_exit(2)
    else: hard_exit(2)
```

pre_check() — environment

- Python `< 3.10` → error & abort (`core.py:95`)
- `curl` missing on PATH → abort
- `ffmpeg` missing on PATH → abort

common_pre_check() — models + safety

- `pre_check()` on all shared analyser modules (download hashes/sources)
- Re-hashes `content_analyser.py` source; must equal `b14e7b92` (see §10.1)

2.2 Command routing

`route(args)` (`core.py:43`) dispatches on the `command` state item. It first applies an optional system-memory limit, then branches:

Command group	Commands	Behaviour
Processing	<code>run</code> , <code>headless-run</code> , <code>batch-run</code> , <code>benchmark</code>	Require <code>common_pre_check</code> + <code>processors_pre_check</code> ; then launch UI or run job(s)
Downloads	<code>force-download</code>	Iterate every analyser + processor model set; fetch & verify
Job management	<code>job-create/submit/delete/add-step/...</code>	Init jobs dir, then CRUD via <code>job_manager</code>
Job execution	<code>job-run/run-all/retry/retry-all</code>	Sequential execution via <code>job_runner</code>

2.3 conditional_process() — the fork to workflows

Every processing path ultimately reaches `conditional_process()` (`core.py:336`). It runs each processor's `pre_process('output')` gate, then dispatches by target media type:

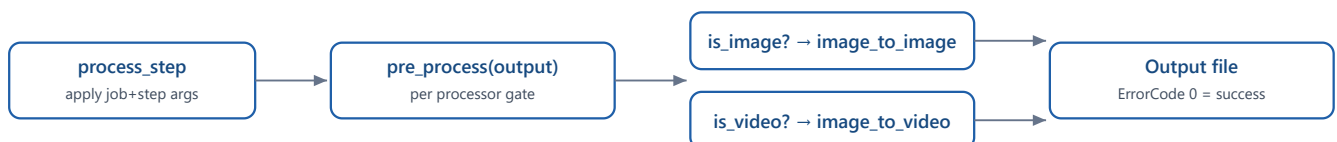


Figure 2 — A single step: arguments are applied to state, each processor's precondition is checked, then the image or video workflow runs.

Error-code convention

Workflows return a small integer `ErrorCode`: `0` success, `1` generic failure, `2` pre-check/validation failure, `3` content-analysis rejection (NSFW), `4` user-initiated stop. `hard_exit(code)` propagates it to the shell.

SECTION 3

Program, Arguments & State

Three modules translate the outside world into runtime configuration: **program.py** defines the CLI, **args.py** bridges parsed arguments to state and to persisted jobs, and **state_manager.py** holds that configuration in two parallel contexts.

3.1 Program composition

program.py builds the CLI from many small, single-topic **ArgumentParser** factories, each created with **add_help=False** so it can act as a reusable *parent*. Two aggregators bundle them:

```
collect_step_program()      detector + landmarker + selector + masker + voice-extractor + frame-extraction + output-creation + processors
collect_job_program()       execution + download-providers + memory + log-level
```

create_program() then registers one sub-parser per command (**run**, **headless-run**, **batch-run**, all job commands, ...), composing each from the relevant parents. Two behaviours are worth noting:

- **Config-backed defaults.** Each argument's **default** comes from **config.get_***, i.e. from **facefusion.ini**, falling back to a hard-coded string.
- **Key registration as a side-effect.** Building a factory also calls **job_store.register_step_keys([...])** or **register_job_keys([...])**. This registry is the single source of truth for which keys are per-step vs job-wide.
- **Dynamic choices.** The detector-size choices depend on the chosen detector model (a preliminary **parse_known_args** reads it first); processors inject their own arguments via **register_args**.

program_helper.validate_args() walks the whole parser tree and verifies that every argument whose **default** is constrained by **choices** actually holds a legal value — catching a misconfigured INI before anything runs.

3.2 Argument ↔ state ↔ job bridging

args.py::apply_args(args, apply_state_item) is the args→state writer. Crucially it takes the state-writer as a *parameter*, so the same code serves two callers:

Caller	Writer passed	Effect
CLI startup (core.py:30)	state_manager.init_item	Seeds <i>both</i> cli & ui buckets from argv
Per-step (core.py:327)	state_manager.set_item	Applies one step's merged args into the active bucket

Four reducer/collector helpers, driven by the **job_store** key registry, move data between argv/state and the persisted job JSON:

```
reduce_step_args(args)      argv → the subset of registered step keys (stored in a job step)
reduce_job_args(args)       argv → the subset of registered job keys (execution, memory, patterns...)
collect_step_args()         live state → step keys (UI snapshots current config into a job)
collect_job_args()          live state → job keys
```

3.3 Dual CLI/UI state

state_manager.py keeps a module-global **STATE_SET = {'cli': {}, 'ui': {}}**. The active bucket is chosen *per call* by **app_context.detect_app_context()**, which walks the call stack and inspects filenames:

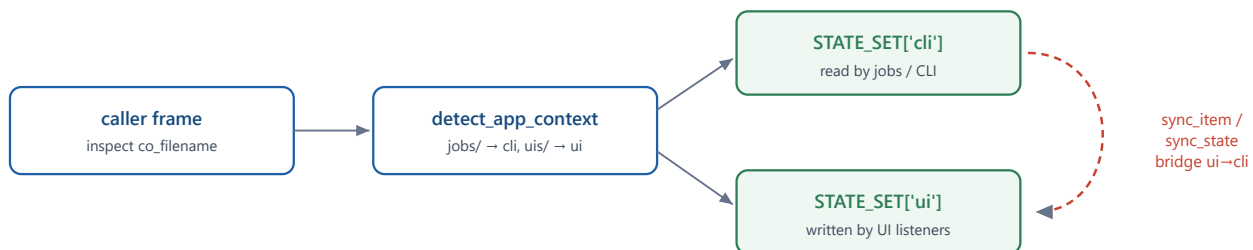


Figure 3 — State context is resolved by call-stack inspection. UI edits touch only the ui bucket until an explicit sync pushes them into cli (where job code reads them).

```
init_item(k,v)              writes to both buckets (startup baseline)
get_item(k) / set_item(k,v) read/write the active bucket only
sync_item(k) / sync_state() copy one key / the whole bucket from ui → cli
```

3.4 Config & choices

config.py

Wraps **facefusion.ini** via a singleton `ConfigParser`. Typed getters (`get_str/int/float/bool_value` + `get_str/int_list`) return the INI value only if present and non-blank, else a cast fallback. Defaults live at the **program.py** call sites, not in the file.

choices.py

The single registry of allowed values: model lists (mostly derived from `Literal` types via `get_args()`), mapping dicts (provider→ORT class, mask region→label ids, encoder sets), and numeric ranges feeding both argparse and UI sliders. Keeps CLI and UI vocabularies identical.

The Face Analysis Pipeline

Before any processor runs, FaceFusion turns raw pixels into structured `Face` objects. This shared pipeline — orchestrated by `face_analyser.py` — runs once per frame and is cached, so every processor reuses the same detections.

4.1 End-to-end face construction

The entry point `get_many_faces()` (`face_analyser.py:96`) checks the frame cache first, then detects across configured angles, then assembles each face:

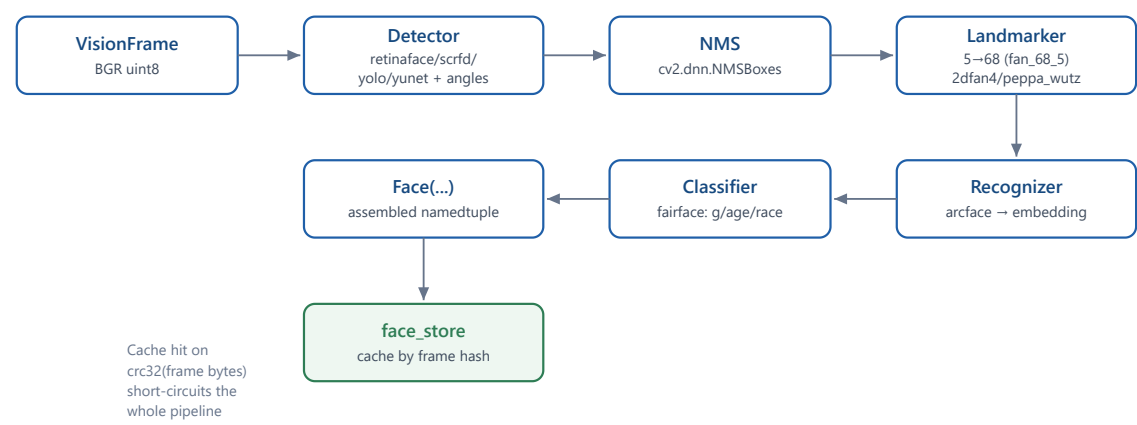


Figure 4 — Per-frame face construction. The 5-point landmark used downstream (`'5/68'`) is upgraded from the 68-point detector output when the landmarker is confident, else the raw detector points.

4.2 The Face type

`Face` is a namedtuple (`types.py:35`) carrying everything downstream processors need:

Field	Type	Meaning
<code>bounding_box</code>	<code>NDArray [x1,y1,x2,y2]</code>	Face box in frame coords
<code>score_set</code>	<code>{detector, landmarker}</code>	Confidence of each stage
<code>landmark_set</code>	<code>{'5','5/68','68','68/5'}</code>	Four landmark variants; <code>'5/68'</code> is the one used for warping
<code>angle</code>	<code>int (0/90/180/270)</code>	Estimated in-plane rotation
<code>embedding</code>	<code>NDArray float</code>	Raw ArcFace identity vector
<code>embedding_norm</code>	<code>NDArray</code>	L2-normalized embedding (for distance)
<code>gender</code>	<code>'male' 'female'</code>	From FairFace
<code>age</code>	Python <code>range</code>	Age bucket, e.g. <code>range(20,30)</code>
<code>race</code>	<code>Literal[6 values]</code>	white/black/latino/asian/indian/arabic

4.3 The analyser modules

Module	Models	Role & key detail
face_detector	retinaface, scrfd, yolo_face, yunet, many	Multi-angle detection; per-model normalization & anchor decode; boxes fed to NMS in the analyser. many runs three detectors together.
face_landmarker	2dfan4, peppa_wutz, fan_68_5	Refines 68-point landmarks; fan_68_5 always lifts 5→68; CLAHE contrast boost for dark faces.
face_recognizer	arcface_w600k_r50	Warpes to a 112 ² ArcFace template, returns raw + L2-normalized embedding.
face_classifier	fairface	224 ² input; predicts gender, age bucket, race.
face_selector	—	Chooses faces by mode (one/many/reference), order, and attribute filters (age/gender/race). Reference mode matches by embedding distance.
face_masker	xseg (occlusion), bisenet (region)	Builds box / occlusion / region / area masks that constrain where a processor writes.
face_store	—	Caches List[Face] keyed by crc32(frame.tobytes()) ; the reason the pipeline runs once per unique frame.

4.4 Selection & masking detail

face_selector modes

- many all faces, sorted & filtered
- one first face after sort/filter
- reference match target faces to a chosen reference face within reference_face_distance (cosine-style distance on embedding_norm)

Order: left-right, right-left, top-bottom, bottom-top, small-large, large-small, best-worst, worst-best.

Filters: gender (exact), race (exact), age (range intersection).

face_masker types

- box geometric; padding + Gaussian blur (no model)
- occlusion XSeg models; hides hands/hair over the face
- region BiSeNet parser; select skin/brows/eyes/nose/mouth/...
- area convex hull over landmark groups (upper/lower face, mouth)

Masks are combined and used at *paste-back* so edits blend only where intended.

SECTION 5

The Processor Plugin System

Processors are the units of transformation. Each is a subpackage under `facefusion/processors/modules/<name>/` whose `core.py` implements a fixed interface. They are loaded dynamically by name and chained per frame.

5.1 Dynamic loading & the contract

The 45-line loader `processors/core.py` imports `facefusion.processors.modules.<name>.core` and validates that it exposes all eight required methods; a missing method raises `NotImplementedError` and hard-exits. `get_processors_modules(names)` returns the validated module objects used throughout the workflows.

Method	Responsibility
<code>get_inference_pool()</code>	Build/return this module's ONNX sessions for the selected model
<code>clear_inference_pool()</code>	Evict this module's cached sessions
<code>register_args(program)</code>	Add CLI args (defaults from config) & register step keys
<code>apply_args(args, writer)</code>	Copy parsed args into state via the injected writer
<code>pre_check()</code>	Ensure model hashes + sources are downloaded & valid
<code>pre_process(mode)</code>	Validate preconditions (e.g. swapper needs a source face; lip-syncer needs audio)
<code>post_process()</code>	Clear caches; under memory strategy <i>strict/moderate</i> , release inference pools
<code>process_frame(inputs)</code>	The core per-frame transformation → returns <code>(frame, mask)</code>

Not part of the interface

`process_image` / `process_video` are *workflow-level* functions, not processor methods. `create_static_model_set()` exists in every module (declaring the ONNX catalogue) but is a private `@lru_cache` d helper, not loader-enforced.

5.2 The process_frame chain

For each frame the workflow builds one `inputs` dict and threads it through *every* selected processor in order; each receives the previous processor's output frame and mask:

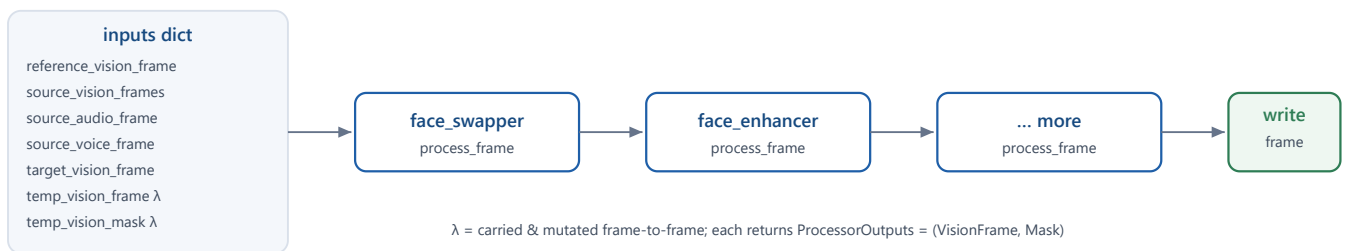


Figure 5 — Processors compose left-to-right on a shared working frame; the mask travels with it and is merged back at the end.

5.3 The eleven processors

Processor	Faces?	Models (examples)	What it does
face_swapper	source+target	inswapper, simswap, ghost, hyperswap, blendswap, uniface, hifface (13)	Identity swap; pixel-boost tiling for super-res; embedding conversion per architecture
face_enhancer	target	codeformer, gfpgan, gpen_bfr, restoreformer++ (9)	Face restoration / super-resolution with blend weight
deep_swapper	target	DeepFaceLive <code>.dfm</code> per-identity + custom (many)	Whole-face swap using pre-trained identity models
face_editor	target	live_portrait (6 sub-models)	Edit eyes, lips, head pose, expression
expression_restorer	target	live_portrait	Restore the target's original expression after edits
age_modifier	target	fran, styleganex_age	Age / de-age a detected face
lip_syncer	target+audio	wav2lip, wav2lip_gan, edtalk	Audio-driven lip synchronization
face_debugger	target	— (uses shared analysers)	Diagnostic overlay: landmarks, boxes, masks, scores
frame_enhancer	no	real_esrgan, clear_reality, swin2_sr, ultra_sharp (20)	Full-frame upscaling (x2–x8) via tiling
frame_colorizer	no	ddcolor, deoldify	Colorize grayscale frames
background_remover	no	rmbg, birefnet, isnet, modnet, u2net (15)	Segment & remove/matte the background

5.4 Shared processor helpers

pixel_boost.py

Two numpy reshape helpers — `implode_pixel_boost` tiles a large face crop into model-sized sub-frames for batched inference, `explode_pixel_boost` reassembles them. Used only by **face_swapper** to swap above the model's native resolution.

live_portrait.py

Motion/expression math for the LivePortrait model family: Euler-angle limits (pitch $\pm 30^\circ$, yaw $\pm 60^\circ$, roll $\pm 20^\circ$), expression clamps, and rotation-matrix construction. Powers **face_editor** and **expression_restorer**.

Model declaration & source keys

Each `create_static_model_set()` returns a nested dict keyed by model name; every entry carries `__metadata__` (vendor/license/year), `hashes`, `sources` (`{url, path}` pairs), plus `template`, `size`, and often `type` / `mean` / `std`. A single model may declare *multiple* source keys — e.g. **face_swapper**'s `face_swapper` + `embedding_converter`, or **face_editor**'s six LivePortrait sub-models — each becoming its own `InferenceSession`.

Inference & Execution Layer

This layer turns model files into running ONNX sessions on the best available hardware, and guarantees model integrity through on-demand download with checksum verification.

6.1 ONNX session pooling

`inference_manager.py` caches sessions in a two-tier global `INFERENCE_POOL_SET = {'cli': {}, 'ui': {}}`. Each pool is keyed by an *inference context* string that concatenates module name, model names, device id, and the ordered execution providers:

module

model name(s)

device id

execution providers (priority order)

facefusion.face_swapper . inswapper_128 . 0 . cuda . cpu

→ distinct GPU id or provider list ⇒ distinct pool. cli→ui pools are cross-shared so one session serves both contexts.

Figure 6 — Inference-pool cache key. Session load failure is fatal (`fatal_exit(1)`).

- **Lazy & shared.** `get_inference_pool()` waits while `process_manager.is_checking()`, resolves providers (a module may override `resolve_execution_providers`), and cross-populates the cli/ui caches so both contexts reuse one session. For multi-GPU it returns a pool for a *randomly chosen* device id (crude load spread).
- **Eviction.** `clear_inference_pool()` deletes the module's contexts; on **Windows + DirectML** it clears the whole app-context bucket (a driver workaround, since DirectML sessions must be torn down together).
- **No central lock.** The only concurrency gate around pool mutation is the `is_checking()` spin-wait; session thread-safety is handled at `.run` time (§10.2).

6.2 Execution-provider priority

`execution.py` derives available providers from ONNX Runtime and re-sorts them into FaceFusion's canonical priority, then maps each to an `ORT (name, options)` tuple. CPU is always appended last as the universal fallback.

Priority	Provider	Notable options set by <code>create_inference_providers</code>
1	CUDA	<code>device_id</code> , <code>cudnn_conv_algo_search</code> (DEFAULT on GTX 16xx, else EXHAUSTIVE)
2	TensorRT	engine & timing cache paths, <code>trt_builder_optimization_level: 4</code>
3	ROCM	<code>device_id</code>
4	MIGraphX	<code>device_id</code> + optional model cache dir
5	CoreML	<code>SpecializationStrategy: FastPrediction</code> + cache dir
6	OpenVINO	<code>device_type</code> (GPU / GPU.N), <code>precision: FP32</code>
7	QNN	<code>device_id</code> , <code>backend_type: htp</code>
8	DirectML	<code>device_id</code> (deprioritized near the tail)
9	CPU	bare fallback, always last

Device discovery runs `nvidia-smi --query --xml-format` (cached) to populate `ExecutionDevice` records — driver, CUDA version, product name, VRAM total/free, temperatures, utilization. Engine/timing caches live under a version-scoped `.caches/<ort-version>` directory to avoid cross-version collisions.

6.3 Model download & integrity

`download.py` fetches models on demand into `.assets/models/`. URLs are resolved by walking `download_providers` in order — GitHub releases, then Hugging Face (with an `hf-mirror` fallback) — pinging each candidate base URL and using the first reachable one.

Download mechanics

`conditional_download` compares the existing partial size to the remote content-length and resumes via `curl` (5s timeout, 5 retries), driving a `tqdm` bar by polling file growth.

Integrity gate

Each `.onnx` has a sidcar `.hash` (CRC32). `validate_source_paths` re-hashes the file and, if it still fails after (re)download, **deletes the corrupt source**. A source is trusted only when its CRC matches.

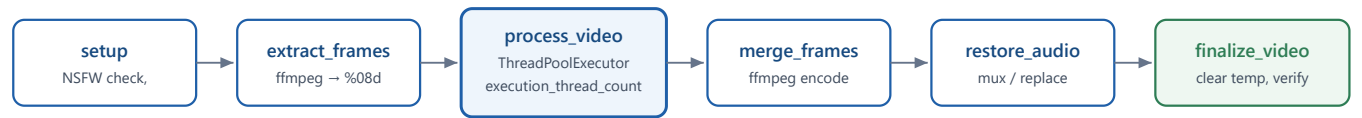
`hash_helper.create_hash()` is simply `crc32` formatted as 8 hex digits — an integrity/ tamper checksum, not a cryptographic hash. The same primitive underpins the content-safety gate (§10.1) and the frame cache key.

Processing Workflows & Media I/O

Two workflow modules orchestrate the full pipeline. The image path is a short linear sequence; the video path adds frame extraction, multi-threaded per-frame processing, re-encoding, and audio restoration.

7.1 The video pipeline

`workflows/image_to_video.py::process()` runs an ordered task list, aborting on the first non-zero error code, bracketed by `process_manager.start()` / `end()`:



A stop request cancels outstanding frame futures; frames are read/written in the per-target temp directory throughout.

Figure 7 — The six-stage video workflow. Only `process_video` is parallel; every other stage is a single FFmpeg or filesystem operation.

Frame parallelism. `process_video` submits every extracted frame to a single `ThreadPoolExecutor(max_workers = execution_thread_count)` (default 8, range 1–32) and consumes results via `as_completed`; a stop request cancels the remaining futures. Each worker runs `process_temp_frame`, which reads the frame (RGBA), fetches the aligned audio/voice frames for lip-sync, runs every processor's `process_frame` in sequence, merges the mask, and writes the frame back in place.

On `execution_queue_count`
Older FaceFusion had a separate queue-count knob; in this version concurrency width is governed solely by `execution_thread_count` — all frames are submitted at once and the executor's internal queue backs them.

7.2 The image pipeline

`image_to_image.py` is the same driver without frames: `setup` (NSFW check + temp dir) → `prepare_image` (resolve output resolution, copy into temp) → `process_image` (read RGBA, run every processor, merge mask, write) → `finalize_image` (encode to output, clean up). RGBA is used so an alpha mask can travel with the frame; processors receive the RGB slice while the mask is carried separately.

7.3 Media I/O

Module	Responsibility
<code>vision.py</code>	Frame read/write with Windows Unicode-path handling (<code>numpy.fromfile</code> + <code>cv2.imdecode</code> instead of <code>cv2.imread</code>); pooled <code>VideoCapture</code> reads under a global semaphore; fps/resolution/duration/trim detection; resize/crop/letterbox/cover; color matching; tiling; mask extract/merge helpers. Single-image reads are <code>lru_cache</code> d (size 64) so worker threads share decoded frames.
<code>ffmpeg_builder.py</code>	Composable command <i>fragments</i> (each returns a token list); <code>chain</code> flattens, <code>concat</code> merges same-flag filters, <code>run</code> prepends the binary + loglevel. Encoders/quality/preset builders map a generic 0–100 quality onto codec-specific CRF/CQ/QP/bitrate and preset names.
<code>ffmpeg.py</code>	Invokes FFmpeg via <code>subprocess.Popen</code> , polling while <code>is_processing()</code> and terminating on stop; high-level ops: <code>extract_frames</code> , <code>merge_video</code> , <code>copy_image</code> / <code>finalize_image</code> , <code>restore_audio</code> / <code>replace_audio</code> , <code>read_audio_buffer</code> , <code>concat_video</code> ; progress parsed from <code>frame=</code> lines into a tqdm bar.
<code>audio.py</code>	Decodes audio via FFmpeg (48kHz/16-bit/stereo), builds an 80-bin mel spectrogram (SciPy STFT), slices into per-frame chunks aligned to video frames; cached. Supplies <code>get_audio_frame</code> / <code>get_voice_frame</code> to lip-sync.
<code>voice_extractor.py</code>	MDX-Net vocal isolation (kim_vocal, uvr_mdynet) via overlap-add STFT framing; isolates the voice track that drives lip-sync.
<code>temp_helper.py</code>	Per-target temp dir <code><temp>/facefusion/<target_name>/</code> holding the working copy + numbered frames; created at setup, cleared at finalize (kept if <code>keep_temp</code>).
<code>process_manager.py</code>	Four-state machine <code>checking</code> / <code>processing</code> / <code>stopping</code> / <code>pending</code> ; workflows bracket with <code>start()</code> / <code>end()</code> ; a stop transition cancels frame futures and terminates FFmpeg.

The Job System

Every run is a *job*: a JSON file of ordered *steps*, each capturing the arguments for one processing operation. This makes interactive, headless, and batch runs share identical execution semantics.

8.1 On-disk model & status flow

A job's status is the directory it sits in under `jobs_path`. Changing status = moving the file. `init_jobs` creates one directory per status.

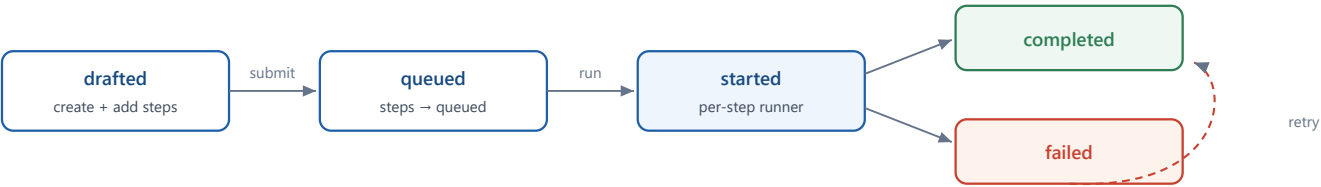


Figure 8 — Job status flow. `retry` resets a failed job's steps to queued and moves it back for another run.

The JSON schema is small: a `Job` has `{version, date_created, date_updated, steps[]}` and each `JobStep` is `{args, status}` where step status adds the transient `started`.

8.2 Manager & runner

job_manager.py (CRUD)

- `create_job` → drafted (only if id unused anywhere)
- `submit_job` → queued (needs ≥1 step)
- `add / insert / remove / remix_step`
- `remix_step` chains a new step onto a prior step's output
- file primitives: read/update (stamps `date_updated`)/move/delete; `find_job_ids` globs sorted by mtime = submission order

job_runner.py (execution)

- `run_job` runs only queued jobs; `run_steps` is sequential and short-circuits on first failure
- `run_step` sets `started`, calls the injected `process_step`, moves the step output
- `finalize_steps` concatenates multi-step video outputs (or moves images)
- `retry_job` re-queues a failed job; success → *completed*, else → *failed*

8.3 Headless & batch as ephemeral jobs

Both non-interactive commands reuse the exact same machinery via a throwaway job:

Command	Behaviour
<code>headless-run</code>	One ephemeral job, one step: <code>create</code> → <code>add_step</code> → <code>submit</code> → <code>run</code> in a single expression
<code>batch-run</code>	One ephemeral job, one step per file: expands <code>source_pattern</code> × <code>target_pattern</code> (cartesian product), formats an <code>output_pattern</code> with <code>{index}</code> / <code>{source_name}</code> / <code>{target_name}</code> / <code>{target_extension}</code>

`job_store.py` holds the `{job_keys, step_keys}` registry populated by `program.py`; `job_helper.py` supplies timestamped ids (`job-YYYY-MM-DD-HH-MM-SS`) and per-step output paths; `job_list.py` renders the status table shown by `job-list` and the UI.

The UI System

The Gradio front-end (**facefusion/uis/**) is a thin, self-wiring layer over the same state and job machinery the CLI uses. Layouts assemble components; components follow a uniform render/listen/update pattern.

9.1 Layouts & loading

uis/core.py dynamically imports a layout module (`facefusion.uis.layouts.<name>`) and asserts it implements the four layout methods `pre_check`, `render`, `listen`, `run`; a missing method hard-exits. Four layouts ship:

default	the full processing UI (~35 components across three columns)
benchmark	performance testing across resolutions
jobs	manage & run persisted jobs from the browser
webcam	live webcam processing

`launch()` builds a `gradio.Blocks` with a custom theme/CSS, calls each layout's `render()` then `listen()` (wrapping in a `Tab` when more than one layout), and finally `run(ui)` which launches the server (with favicon and optional auto-open browser).

9.2 The component pattern

Every component module is a small self-contained unit holding its Gradio widget in a module global and exposing three functions:

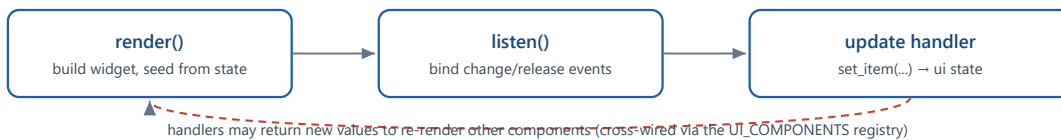


Figure 9 — The render/listen/update cycle. A global component registry lets components in different files observe each other's widgets.

Handlers write via `state_manager.set_item`, which — because the call originates under **uis/** — lands in the *ui* bucket. When the user launches a job, execution flips to the *cli* context, so the `sync_item` / `collect_*` bridges (§3.2–3.3) carry UI edits into the bucket the job runner reads. The job-related components (`instant_runner`, `job_runner`, `job_manager`, `job_list`, `ui_workflow`) are simply a UI surface over the §8 job APIs.

Cross-Cutting Concerns & Appendices

10.1 The content-safety gate

Hash-pinned NSFW detector — do not edit `content_analyser.py`

`core.py::common_pre_check()` re-hashes the source of `content_analyser.py` with `crc32` and aborts startup unless it equals the literal `b14e7b92`. This runs before `run`, `headless-run`, `batch-run`, and `benchmark`. **Any change to that file — even whitespace — changes the hash and makes every processing command hard-exit.** If a refactor is unavoidable, the expected hash in `common_pre_check` must be updated to match.

Functionally, the content analyser is a three-model NSFW ensemble (EraX, Marqo, Freepik). A frame is flagged only on a **2-of-3 majority**; a video is rejected when the positive-detection rate exceeds 10% of sampled frames (one frame per second). This is a deliberate safety mechanism, not incidental code.

10.2 Threading & concurrency model

Where parallelism lives

Frame-level: one

`ThreadPoolExecutor(execution_thread_count)` in `process_video`. Within a frame, processors run sequentially. Multi-GPU spread is a random device-id choice per pool acquisition.

How correctness is preserved

`thread_semaphore()` (a global count-1 semaphore) serializes non-thread-safe OpenCV capture and some ONNX runs. `conditional_thread_semaphore()` narrows this to fragile backends only — **DirectML (Win)**, **MIGraphX / ROCm (Linux)** — letting CUDA/CPU run inference concurrently.

10.3 Caching summary

Cache	Key	Purpose
face_store <code>static_faces</code>	<code>crc32(frame bytes)</code>	Skip the whole face pipeline for repeated frames
inference pools	module+models+device+providers	Reuse ONNX sessions across contexts
<code>create_static_model_set</code>	<code>lru_cache</code> (download scope)	Build model catalogues once
<code>read_static_image/video/audio</code>	<code>lru_cache(64)</code>	Share decoded media across worker threads
ping / device detection	<code>lru_cache</code>	Avoid repeated network/nvidia-smi calls

10.4 Memory management

`memory.py::limit_system_memory()` caps the process working set — via `SetProcessWorkingSetSize` on Windows, `setrlimit(RLIMIT_DATA)` on POSIX. GPU memory is governed separately by the `video_memory_strategy` option (*moderate/strict*), which drives how aggressively processors clear inference pools in `post_process`. `OMP_NUM_THREADS=1` is set for ONNX stability.

10.5 Development & CI

Lint	<code>flake8 facefusion.py install.py facefusion tests</code> — tabs, import order (flake8-import-order, pycharm style)
Types	<code>mypy ...</code> — full annotations required (<code>disallow_untyped_defs/calls</code> , <code>disallow_any_generics</code>)
Tests	<code>pytest</code> / <code>pytest tests --cov facefusion</code>
CI	Python 3.12 on macOS / Ubuntu / Windows; the lint job gates the tests
Install	<code>python install.py --onnxruntime {default cuda directml tensorrt openvino rocm migraphx}</code>

10.6 Key types (types.py)

Face	the namedtuple of §4.2
VisionFrame	NDArray — BGR uint8 image/video frame
Mask	NDArray alpha mask carried through the processor chain
ProcessorOutputs	Tuple[VisionFrame, Mask] returned by <code>process_frame</code>
InferencePool / ...Set	model_name→session; app_context→context→pool
ProcessState	'checking' 'processing' 'stopping' 'pending'
Job / JobStep	persisted job schema of §8.1
ExecutionProvider	the Literal that defines provider priority order

This document was generated from a direct reading of the FaceFusion source tree (v3.6.1). Line references reflect the state of the repository at analysis time and may drift as the code evolves; treat **file.py:line** pointers as starting points rather than exact addresses. FaceFusion is licensed OpenRAIL-AS — review the license before any commercial or redistributive use of the platform or its models.